Unit 3 Progress Check: FRQ Part B

1. The Car class, which you will write, is used to represent information about a car.

Car objects are created by calls to a constructor with two double parameters. The first parameter represents the number of gallons of fuel in the car. The second parameter represents the average number of miles the car can travel per gallon of fuel.

The Car class contains a canMakeTrip method, which determines whether there is enough fuel in the car to make a trip. This method has a double parameter that represents the distance, in miles, of a potential trip. If there is enough fuel, the method updates the current amount of fuel in the car and returns true. If there is not enough fuel for the entire distance of the trip, the amount of fuel is unchanged and the method returns false.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Car.

Statement	Return Value (blank if no value)	Explanation
boolean possible;		
Car carOne = new Car(10.0, 30.0);		A Car object named CarOne is constructed with 10 gallons of fuel and an average of 30 miles per gallon.
possible = carOne.canMakeTrip(120.0);	true	At 30 miles per gallon, a 120-mile trip will require 4 gallons of fuel. Since this is less than or equal to the 10 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 6 gallons.
possible = carOne.canMakeTrip(90.0);	true	At 30 miles per gallon, a 90-mile trip will require 3 gallons of fuel. Since this is less than or equal to the 6 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 3 gallons.

Unit 3 Progress Check: FRQ Part B

Statement	Return Value (blank if no value)	Explanation
possible = carOne.canMakeTrip(225.0);	false	At 30 miles per gallon, a 225-mile trip will require 7.5 gallons of fuel. Since this is more than the 3 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
Car carTwo = new Car(12.0, 15.0);		A Car object named carTwo is constructed with 12 gallons of fuel and an average of 15 miles per gallon.
possible = carTwo.canMakeTrip(300.0);	false	At 15 miles per gallon, a 300-mile trip will require 20 gallons of fuel. Since this is more than the 12 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
possible = carTwo.canMakeTrip(180.0);	true	At 15 miles per gallon, a 180-mile trip will require 12 gallons of fuel. Since this is less than or equal to the 12 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 0 gallons.

Write the complete `Car` class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class Car`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
```

Unit 3 Progress Check: FRQ Part B

```
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class
- Include () with or without parameters in the class header

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables for the current fuel and miles per gallon

Canonical Solution:

Unit 3 Progress Check: FRQ Part B

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly initialize the instance variables

Responses will not earn the point if they

- declare any instance variable `static`
- omit `private` in instance variable declaration
- declare a variable outside the class
- declare an instance variable inside the constructor

Point 3

Unit 3 Progress Check: FRQ Part B

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `Car(double ___, double ___)`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private` or `static`

Point 4

Unit 3 Progress Check: FRQ Part B

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns correct initial values to all instance variables

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- write a class header with parameter and assign the parameter to the instance variables

Responses will not earn the point if they

Unit 3 Progress Check: FRQ Part B

- fail to declare or initialize instance variable for capacity
- assign values to instance variables from an unknown source
- assigns initial value to local variable instead of instance variable, even if the local variables are declared as `private`

Point 5

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `boolean canMakeTrip(double ___)`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```



0	1

Unit 3 Progress Check: FRQ Part B

The response earns the point.

Responses will not earn the point if they

- declare method as something other than `public`
- use incorrect method name
- use incorrect return or parameter type

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines whether there is enough fuel for the trip and updates fuel when appropriate (*algorithm*)

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

Unit 3 Progress Check: FRQ Part B

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to return the `boolean` value
- use a variable declared outside the method as an instance variable when instance variables do not exist
- declare instance variables outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to update the instance variable(s)
- determine the correct result but return something else

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns determined `boolean` value for whether the trip can be made

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
```

Unit 3 Progress Check: FRQ Part B

```
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- return an incorrect value

Responses will not earn the point if they

- fail to return a value in some cases

Unit 3 Progress Check: FRQ Part B

2. The TrashCan class, which you will write, represents a trash can.

TrashCan objects are created by calls to a constructor with a double parameter that represents the trash can's capacity, which is the maximum weight, in pounds, that a TrashCan object can hold. Assume that this value will be greater than or equal to 0. When a Trashcan object is constructed, it is initially empty and its contents have a weight of 0 pounds.

The TrashCan class contains an acceptTrash method, which has a double parameter that represents the weight, in pounds, of the trash that will be deposited into the trash can. The amount to be deposited will always be greater than 0 and less than the capacity of the can. If there is already trash in the trash can, the amount to be deposited, combined with the existing trash, may reach or exceed the can's capacity. In this case, the trash can will be filled to its capacity and then emptied before the remaining trash is accepted. The trash can should always be emptied when the weight of the trash in the can reaches the can's capacity. The acceptTrash method returns a double that represents the number of pounds of trash that can still be added to the trash can before it is completely filled.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than TrashCan.

Statement	Return Value (blank if no value)	Explanation
double remaining;		
TrashCan kitchen = new TrashCan(10.0);		A TrashCan object named kitchen is constructed with a capacity of 10 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = kitchen.acceptTrash(2.5);	7.5	kitchen now holds 2.5 pounds of trash and can take 7.5 more pounds before it needs to be emptied.
remaining = kitchen.acceptTrash(3.5);	4.0	kitchen now holds 6 pounds of trash and can take 4 more pounds before it needs to be emptied.
remaining = kitchen.acceptTrash(6.0);	8.0	kitchen accepts 4 pounds of trash, reaching the 10-pound capacity, and is then emptied. The remaining 2 pounds of trash are then accepted. kitchen now holds 2 pounds of

Unit 3 Progress Check: FRQ Part B

Statement	Return Value (blank if no value)	Explanation
		trash and can take 8 more pounds before it needs to be emptied.
TrashCan bedroom = new TrashCan(3.0);		A TrashCan object named bedroom is constructed with a capacity of 3 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = bedroom.acceptTrash(1.0);	2.0	bedroom now holds 1 pound of trash and can take 2 more pounds before it needs to be emptied.
remaining = bedroom.acceptTrash(3.2);	1.8	bedroom accepts 2 pounds of trash to reach the 3-pound capacity and then is emptied. The remaining 1.2 pounds of trash are then accepted. bedroom now holds 1.2 pounds of trash and can take 1.8 more pounds before it needs to be emptied.

Write the complete TrashCan class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class TrashCan`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
```

Unit 3 Progress Check: FRQ Part B

```
      capacity = weightCapacity;
      trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
    }
}
```



0	1

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class
- Include () with or without parameters in the class header

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables that can be used to determine the trash weight and capacity of the trash can

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;
```

Unit 3 Progress Check: FRQ Part B

```
    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly initialize the instance variables

Responses will not earn the point if they

- declare any instance variable `static`
- omit `private` in instance variable declaration
- declare a variable outside the class
- declare an instance variable inside the constructor

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `TrashCan(double ___)`

Unit 3 Progress Check: FRQ Part B

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private` or `static`

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns correct initial values to capacity instance variable

Unit 3 Progress Check: FRQ Part B

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- write a class header with parameter and assign the parameter to the instance variable

Responses will not earn the point if they

- fail to declare or initialize instance variable for capacity
- assign a value to instance variables from an unknown source
- assign initial value to local variable instead of instance variable, even if the local variable is declared as `private`

Point 5

Unit 3 Progress Check: FRQ Part B

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `double acceptTrash(double ___)`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare method as something other than `public`
- use incorrect method name
- use incorrect return or parameter type

Unit 3 Progress Check: FRQ Part B

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Correctly calculates and updates variable used to maintain amount of trash (or space remaining) in the trash can, including emptying when appropriate (*algorithm*)

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to return the correct amount

Unit 3 Progress Check: FRQ Part B

- use a variable declared outside the method as an instance variable when instance variables do not exist
- declare instance variables outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to update the instance variable(s)
- calculate the correct remaining capacity but return something else

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns calculated amount remaining from within `acceptTrash`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
   }
}
```

Unit 3 Progress Check: FRQ Part B

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- return an incorrect amount

Responses will not earn the point if they

- fail to return a value in some cases